SISTEMA DE ACOMPANHAMENTO DE CICLOS MENSTRUAIS

Anna Luisa Ferreira, Erik Matheus, Giovanna Siqueira, Lethicia Hikary

INTRODUÇÃO

O trabalho apresenta a análise e os testes de software do Ciclo+, um sistema de acompanhamento do ciclo menstrual. O objetivo é avaliar suas funcionalidades, identificar possíveis falhas e verificar oportunidades de melhoria, contribuindo para a qualidade e a confiabilidade da aplicação.

ESCOPO

O produto consiste em um banco de dados para um sistema de acompanhamento menstrual, destinado ao registro e organização de informações relacionadas aos ciclos menstruais.
O banco de dados permitirá armazenar informações referentes aos ciclos, fluxo menstrual, sintomas, humores, medicamentos utilizados e fases do ciclo, mantendo essas informações organizadas para consultas posteriores.

Principais grupos de interesse (Stakeholders)

- Usuários; Pessoa que possui seus dados e registros de ciclos armazenados no sistema.
- Equipe de Desenvolvimento; Responsáveis pela criação e estruturação do banco de dados.
- Equipe de testes/qualidade;
  Responsáveis pela verificação do funcionamento do banco de dados.

HISTORIAS DE USUÁRIO

Lydia aos 18 anos descobriu uma síndrome do ovário policístico e todo mês acaba sofrendo despreparada com suas cólicas. Ela até anota as datas de início da sua menstruação, mas acaba perdendo os papéis.

“Como mulher na fase reprodutiva, eu quero conseguir acompanhar meu período e medicamentos para que eu sempre esteja preparada para as cólicas.”

Mariana com 38 anos deseja ser mãe, contudo, está tendo dificuldades.
Para maximizar suas chances ela procura anotar o início dos seus ciclos menstruais para fazer a cota de quando estará em seu período fértil.
“Como mulher que busca engravidar, eu quero poder anotar todas as fases do meu ciclo para que meu objetivo seja completo.
”

Funcionalidades

Principais recursos do Ciclo+, expostos por uma API REST com persistência em banco de dados

01
Cadastro de usuários
CRUD completo no recurso /usuarios (POST, GET, PUT, DELETE), armazenando nome, data de nascimento e identificador único (idUsuario).

02
Ciclos menstruais
Registro das datas de início e término de cada ciclo, com regra que impede data final anterior à data inicial.

03
Registro de sintomas
Descrição e intensidade vinculadas a um ciclo; o sistema bloqueia duplicidade (HTTP 409) e dados incompletos (HTTP 400).

04
Humores e medicamentos
Registro dos estados emocionais e dos medicamentos utilizados, associados ao ciclo correspondente.

05
Fases do ciclo
Organização das fases menstrual, folicular, ovulatória e lútea, apoiando a identificação do período fértil.

06
Consulta de histórico
Dados persistidos e disponíveis para consultas posteriores, permitindo acompanhar a evolução dos ciclos.

TESTES REALIZADOS

- PMD (Programming Mistake Detector)
- Testes manuais
- Testes de API

Utilizamos quatro ferramentas para quatro testes diferentes; PMD, Postman, JUnit e Selenium.
Ademais, antes de automatizar os testes fizemos testes manuais no software.

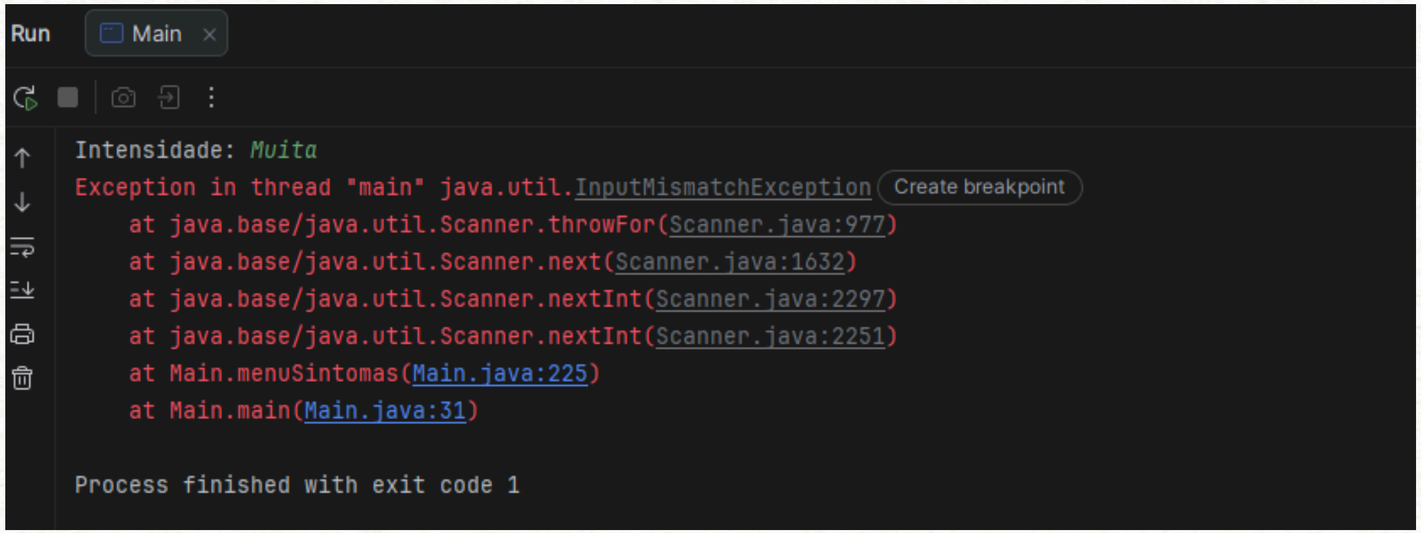
PMD (Programming Mistake Detector)

```
C:\Users\Usuario>pmd check C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src -R rulesets/java/quickstart.xml
[WARN] Progressbar rendering conflicts with reporting to STDOUT. No progressbar will be shown. Try running with argument -r <file> to output the report to a file instead.
[WARN] This analysis could be faster, please consider using Incremental Analysis: https://docs.pmd-code.org/pmd-doc-7.27.0/pmd_userdocs_incremental_analysis.html
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\Ciclo.java:3:  NoPackage:      All classes, interfaces, enums and annotations must belong to a named package
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\CicloMenstrual.java:3:NoPackage:       All classes, interfaces, enums and annotations must belong to a named pac
kage
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\Main.java:5:   NoPackage:      All classes, interfaces, enums and annotations must belong to a named package
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\Main.java:9:   LooseCoupling:  Avoid using implementation types like 'ArrayList'; use the interface instead
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\Main.java:10:  LooseCoupling:  Avoid using implementation types like 'ArrayList'; use the interface instead
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\Main.java:11:  LooseCoupling:  Avoid using implementation types like 'ArrayList'; use the interface instead
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\Main.java:12:  LooseCoupling:  Avoid using implementation types like 'ArrayList'; use the interface instead
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\Main.java:28:  NonExhaustiveSwitch:    Switch statements or expressions should be exhaustive, add a default case
 (or missing enum branches)
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\Main.java:52:  NonExhaustiveSwitch:    Switch statements or expressions should be exhaustive, add a default case
 (or missing enum branches)
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\Main.java:111: NonExhaustiveSwitch:    Switch statements or expressions should be exhaustive, add a default case
 (or missing enum branches)
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\Main.java:173: NonExhaustiveSwitch:    Switch statements or expressions should be exhaustive, add a default case
 (or missing enum branches)
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\Main.java:235: NonExhaustiveSwitch:    Switch statements or expressions should be exhaustive, add a default case
 (or missing enum branches)
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\Sintomas.java:1:      NoPackage:       All classes, interfaces, enums and annotations must belong to a named pac
kage
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\Usuario.java:3:       NoPackage:       All classes, interfaces, enums and annotations must belong to a named pac
kage
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\Usuario.java:7:       UnusedPrivateField:      Avoid unused private fields such as 'duracaoCiclo'.
C:\Users\Usuario\Downloads\Sistema-de-Acompanhamento-master\Sistema-de-Acompanhamento-master\src\Usuario.java:8:       UnusedPrivateField:      Avoid unused private fields such as 'duracaoMenstruacao'.
[INFO] Found 16 violations.

C:\Users\Usuario>
```

Toda essa análise se resume a erros sintáticos presentes no código, apontando a linha do erro e no final a quantidade de violações encontradas

TESTES MANUAIS



Nos testes manuais primeiramente, foi observado que, quando há uma falha, o programa para.
Essa parada acontece com informações inseridas de forma errada.

TESTES MANUAIS

```
ID: 11111111111
Exception in thread "main" java.util.InputMismatchException Create breakpoint : For input string: "11111111111"
    at java.base/java.util.Scanner.nextInt(Scanner.java:2303)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2251)
    at Main.menuUsuario(Main.java:60)
    at Main.main(Main.java:29)

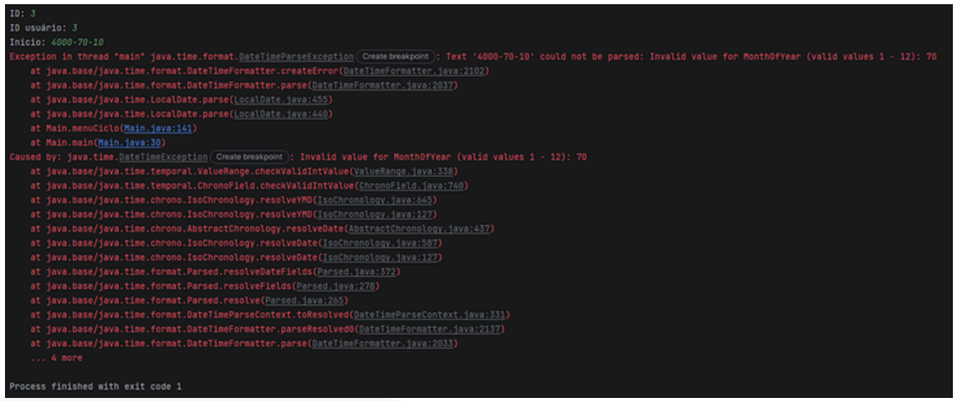
Process finished with exit code 1
```

O método “getIdUsuario” recebe somente até números de 10 digitos. Programa deu parada esperada

```
Nascimento: 111
Exception in thread "main" java.time.format.DateTimeParseException Create breakpoint : Text '111' could not be parsed at index 0
    at java.base/java.time.format.DateTimeFormatter.parseResolved0(DateTimeFormatter.java:2131)
    at java.base/java.time.format.DateTimeFormatter.parse(DateTimeFormatter.java:2033)
    at java.base/java.time.LocalDate.parse(LocalDate.java:455)
    at java.base/java.time.LocalDate.parse(LocalDate.java:440)
    at Main.menuUsuario(Main.java:67)
    at Main.main(Main.java:29)

Process finished with exit code 1
```

Método “getDataNascimento” apresenta o formato ‘ano-mês-dia’, portanto qualquer dado inserido que fuja disso, é terminado o programa com erro. Programa deu parada esperada.



Método “getDataInicio” não permite datas imaginárias, que fujam do tempo atual. Programa deu parada esperada.

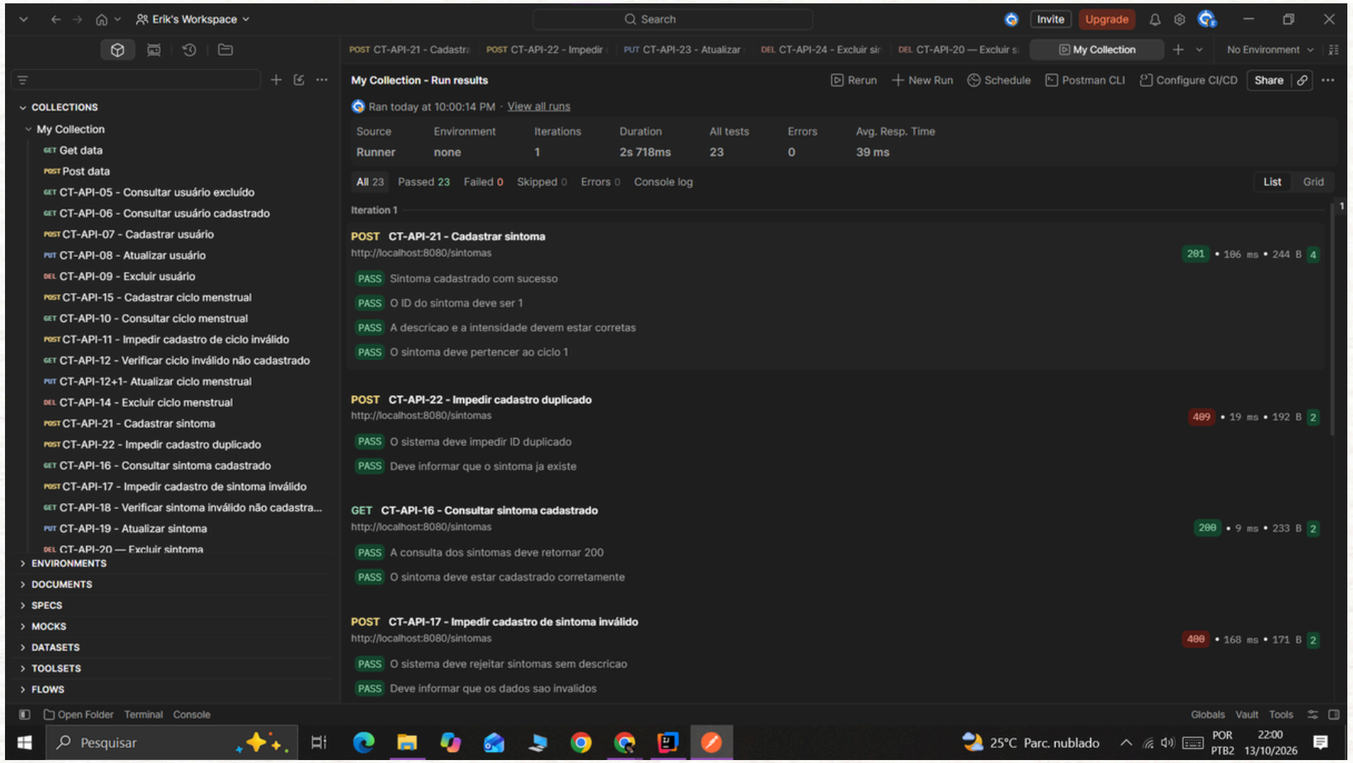
```
===== MENU PRINCIPAL =====
1 - Usuário
2 - Ciclo Mensal
3 - Sintomas
4 - Ciclo Menstrual
0 - Sair
0

Process finished with exit code 0
```

Programa apresentou saída esperada ao escolher sair do programa.

API (Application Programming Interface)

Testes automatizados da API REST com Postman – execução da coleção completa (Collection Runner)

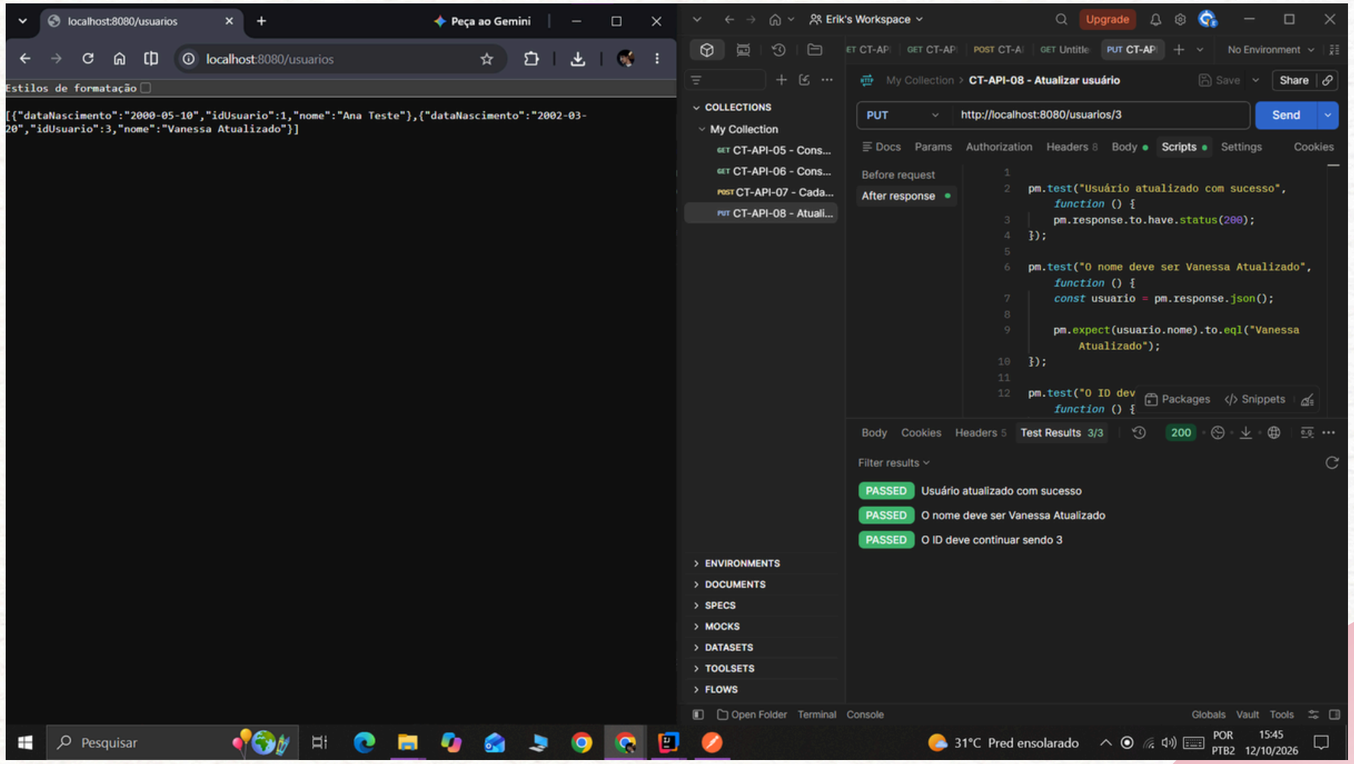


- **Ambiente:** API executada localmente em localhost:8080, com requisições organizadas na coleção “My Collection”.
- **Cobertura:** casos CT-API-05 a CT-API-24 para usuários, ciclos e sintomas, usando os métodos GET, POST, PUT e DELETE.
- **Status HTTP:** 201 (criado), 200 (consulta), 409 (registro duplicado) e 400 (dados inválidos) validados em cada caso.
- **Asserções:** scripts pm.test verificam o corpo da resposta: ID, descrição, intensidade e vínculo com o ciclo.
- **Resultado:** 23 testes executados, 23 aprovados, 0 falhas; tempo médio de resposta de 39 ms.

API (Application Programming Interface)

Caso de teste CT-API-08 – Atualizar usuário (método PUT)

- Requisição: PUT /usuarios/3 substitui os dados do recurso existente; operação idempotente.
- Script de teste: na aba “After response”, pm.test e pm.expect (sintaxe Chai) avaliam a resposta.
- Validações: status 200, nome igual a “Vanessa Atualizado” e ID mantido como 3 (chave primária preservada).
- Resultado: Test Results 3/3 – todas as asserções aprovadas.
- Prova de persistência: GET /usuarios no navegador retorna o JSON com o registro já atualizado no banco.



JUnit

Framework de código aberto da família xUnit para criar e executar testes automatizados de pequenas unidades de código Java, de forma isolada e repetível.

01
Arquitetura JUnit 5

JUnit Platform (motor de execução), JUnit Jupiter (API de escrita dos testes) e JUnit Vintage (compatibilidade com JUnit 3 e 4).

02
Anotações

@Test marca o método de teste; @BeforeEach e @AfterEach preparam e limpam o cenário; @DisplayName e @ParameterizedTest variam entradas.

03
Asserções

assertEquals, assertTrue, assertFalse, assertNull e assertThrows comparam o resultado obtido com o esperado; uma falha é registrada no relatório.

04
Integração

Execução pela IDE (IntelliJ), por Maven/Gradle (Surefire) e em pipelines de CI, seguindo o padrão AAA: Arrange, Act, Assert.

TESTE UNITÁRIO

ValidadorCiclo.java – classe testada

```java
import java.time.LocalDate;

public class ValidadorCiclo {

  public static boolean validarDatas(LocalDate dataInicio,
      LocalDate dataFinal) {
    if (dataInicio == null || dataFinal == null) {
      return false;
    }

    return !dataFinal.isBefore(dataInicio);
  }
}
```

- Unidade testada: validarDatas(LocalDate, LocalDate), método estático e sem efeitos colaterais.
- Regra de negócio: retorna false se alguma data for null (evita NullPointerException) e true apenas quando a data final não é anterior à inicial (isBefore).

TesteValidadorCiclo.java – cenários de teste

```java
import java.time.LocalDate;

public class TesteValidadorCiclo {

  public static void main(String[] args) {
    boolean resultadoValido = ValidadorCiclo.validarDatas(
      LocalDate.of(2026, 10, 1),
      LocalDate.of(2026, 10, 5)
    );

    boolean resultadoInvalido = ValidadorCiclo.validarDatas(
      LocalDate.of(2026, 10, 5),
      LocalDate.of(2026, 10, 1)
    );

    System.out.println("Datas válidas: " + resultadoValido);
    System.out.println("Datas inválidas: " + resultadoInvalido);
  }
}
```

- Cenários: válido 01/10/2026 → 05/10/2026 retorna true; inválido 05/10/2026 → 01/10/2026 retorna false.
- Evolução: migrar o main() para métodos @Test do JUnit 5 com assertTrue/assertFalse e cobrir casos de borda (datas iguais e nulas).

RESULTADOS

16

violações de boas práticas apontadas pelo PMD

1

exceção não tratada nos testes manuais

23/23

testes de API aprovados no Postman

39 ms

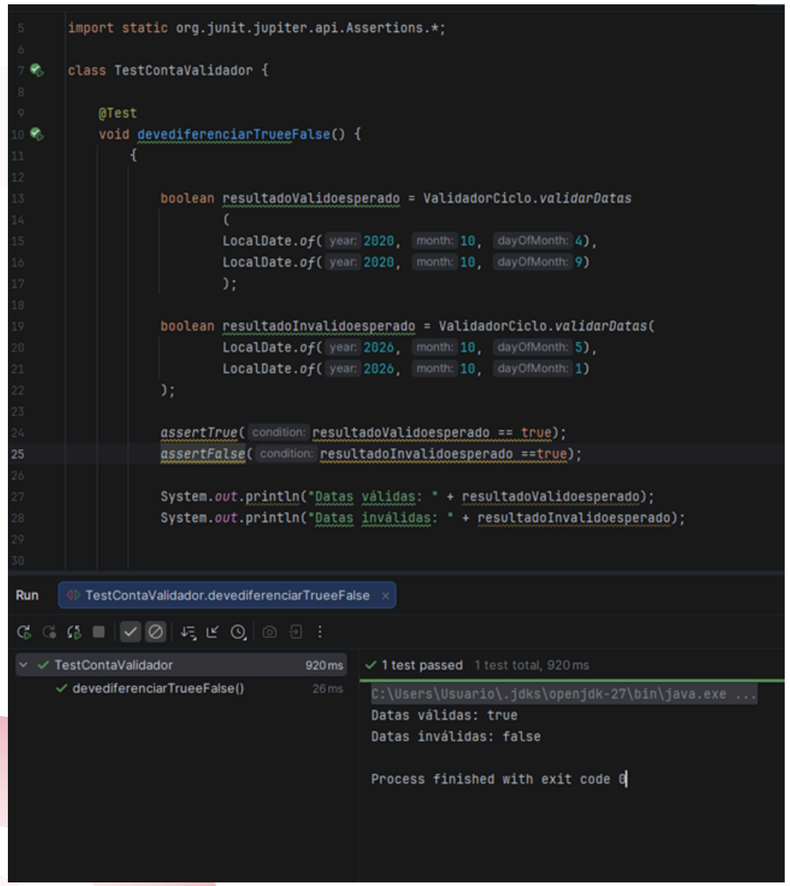
tempo médio de resposta da API

- Análise estática (PMD): 5 NoPackage, 5 NonExhaustiveSwitch, 4 LooseCoupling e 2 UnusedPrivateField – problemas de qualidade e manutenção, que não impedem a compilação.
- Testes manuais: entrada textual (“Muito”) em campo numérico lança InputMismatchException em Scanner.nextInt (Main.menuSintomas) e encerra o programa com exit code 1.

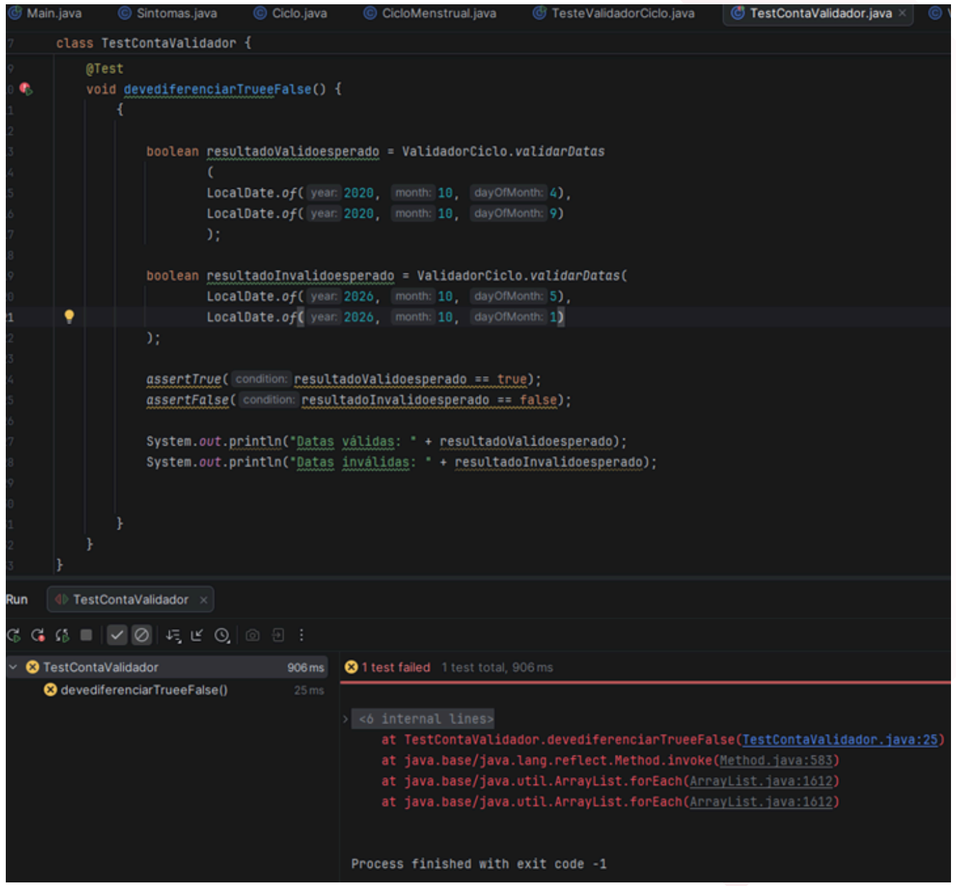
- Testes de API: CRUD de usuários, ciclos e sintomas validado; códigos 200, 201, 400 e 409 retornados conforme o esperado e persistência confirmada.
- Teste unitário: a regra de datas do ciclo respondeu corretamente aos cenários válido e inválido.

RESULTADOS

Caso aceitável:



Caso não-aceitável:



CRITÉRIOS DE AVALIAÇÃO

Instruções

Item Avaliado	Valor	Subcritérios	Visto	Nota
1. Documento de Visão	0,25	Clareza na definição do problema e objetivos do sistema		
		Identificação dos stakeholders e suas necessidades		0,25
		Delimitação clara do escopo do sistema		
2. Histórias de Usuário das Funcionalidades	0,25	Uso correto do formato padrão de história de usuário (Como [usuário], quero [...], para [...])		
		Inclusão das funcionalidades principais		0,25
		Definição de critérios de aceitação claros		
3. Planejamento de Testes	0,5	Seleção adequada das funcionalidades		
		Estratégia de teste compatível com o sistema (No mínimo três)		0,5
		Alinhamento com os requisitos		
4. Descrição dos Casos de Teste	0,5	Estrutura clara dos casos (cenário, entradas, ações, resultado esperado)		
		Uso de Ferramentas (Cypress, Selenium, Postman, Carga etc).		0,5
		Teste e Execução (on-line) - Três testes no mínimo.		
			Total:	1,5

1. Documento de Visão

Peso: 0,25

Problema, objetivo e escopo definidos na Introdução e no Escopo; stakeholders (usuários, desenvolvimento e QA) identificados.

2. Histórias de Usuário

Peso: 0,25

Formato “Como [usuário], eu quero [...], para que [...]” aplicado às personas Lydia e Mariana.

3. Planejamento de Testes

Peso: 0,5

Quatro estratégias compatíveis com o sistema: análise estática (PMD), testes manuais, testes de API e teste unitário.

4. Casos de Teste

Peso: 0,5

Casos CT-API com cenário, entrada, ação e resultado esperado, executados no Postman com evidências.

CONCLUSÃO

Pontos fortes

- API estável: 50/50casos aprovados, 0 erros e tempo médio de resposta de 39 ms.
- Regras de negócio protegidas: rejeição de ciclos inválidos, sintomas duplicados (409) e dados incompletos (400).
- Lógica validada: a verificação de datas do ciclo passou nos cenários válido e inválido.

Oportunidades de melhoria

- Tratamento de entrada: capturar InputMismatchException (try/catch ou hasNextInt()) para o programa não encerrar.
- Qualidade do código: corrigir as 16 violações do PMD – declarar pacotes, usar List, incluir default nos switch e remover campos sem uso.
- Automação: migrar os testes para JUnit 5 e executar a coleção via Postman CLI/Newman em um pipeline de CI.

A combinação de análise estática, testes manuais e testes automatizados aumentou a confiabilidade do Ciclo+ e definiu com clareza as próximas melhorias.